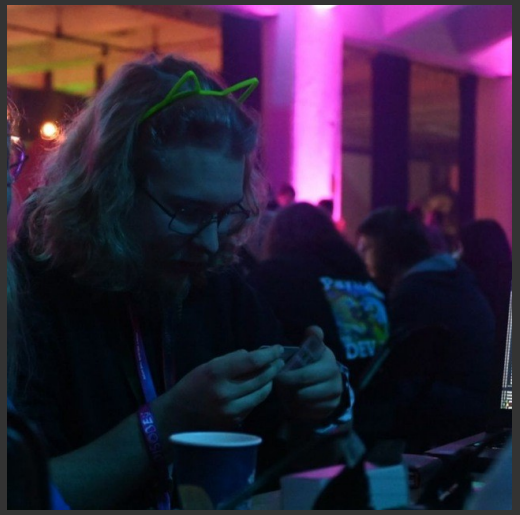
NFC/RFID Hacking 101

~$ whoami

- Santtu “Sanduuz” Sievänen
- Security Specialist @ DNV Cyber
- Hobbies
  - NFC/RFID technologies
  - CTF competitions
  - Music
  - Hiking



Agenda

- Terminology
- Access Control Systems
- Background information
- Examples of HF cards
- Hands-on challenges

As this is a workshop, feel free to ask questions :)

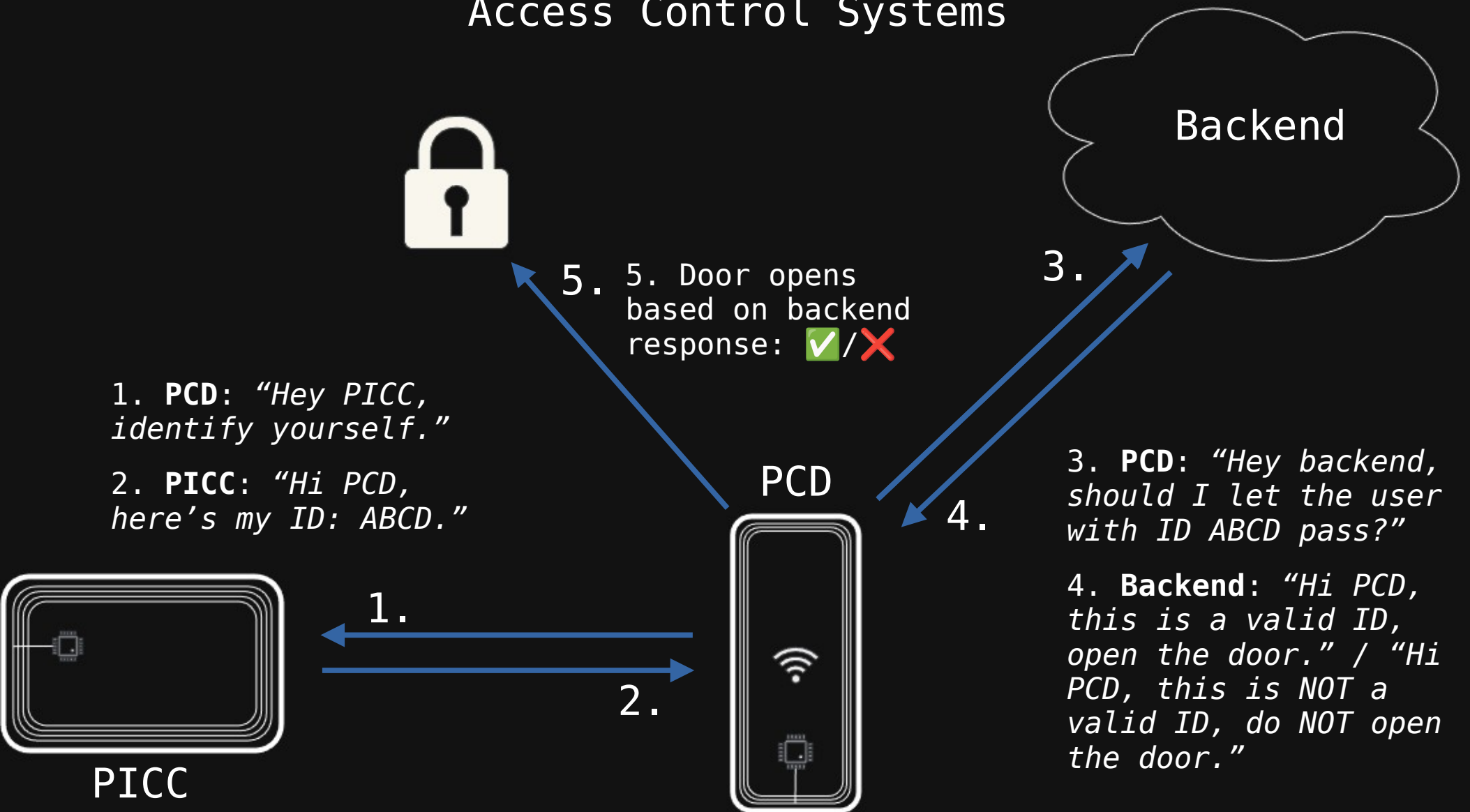
Terminology

- RFID – Radio Frequency Identification
  - Umbrella term
  - LF, HF, UHF
- NFC – Near Field Communication
  - Only HF (13.56 MHz)
  - Defined mainly by ISO/IEC 14443 and ISO/IEC 18092 standards

Terminology

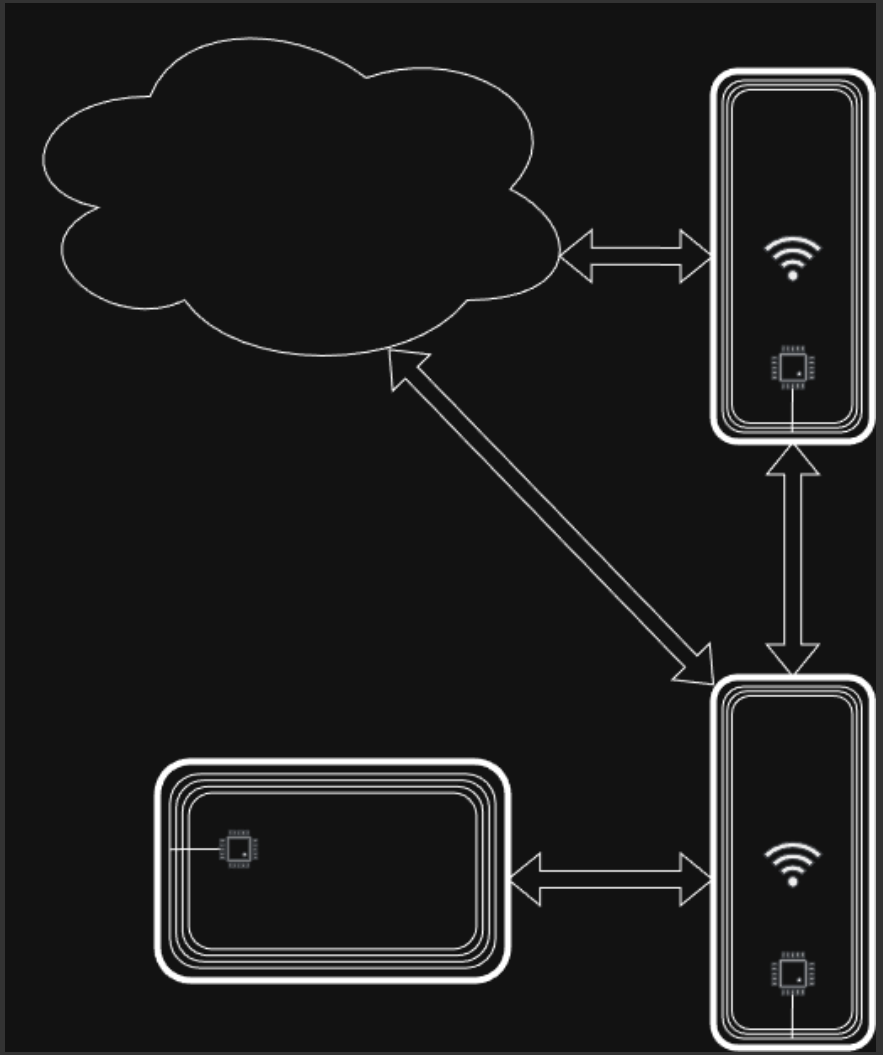
- PICC – **P**roximity **I**ntegrated **C**ircuit **C**ard
- PCD – **P**roximity **C**oupling **D**evice (Reader)
- RF – **R**adio **F**requency
- UID – **U**nique **ID**entifier; 4, 7 or 10 bytes

Access Control Systems



Attack targets

- PICC (E.g. weak crypto)
- PCD (E.g. dumping keys from memory)
- Connection between card and reader (Sniffing)
- Connection between reader and backend systems (API/backend testing)
- Connections between PCD’s (OSDP)
- Protocols (E.g. relay attacks)



Researching keycards

- A PCD and/or a PICC is needed
- Configure the PICC for legitimate use first to familiarize with the specific card technology
- Lots of reading
  - Documentation*
  - Other people’s research (Whitepapers)

* Full documentation might not always be available without an NDA. However, some documentation is available in order for vendors to be able to use the card technology.

Low Frequency (125 kHz)

- Rule of thumb: lower frequency = slower data exchange rate
- No cryptography (= usually easy to clone*)
- UID/Facility Code & Card Number based authentication
- Examples: EM4100, T5577, HID Indala Prox

* Sometimes harder due to a proprietary communication protocol (security through obscurity)

High Frequency (13.56 MHz)

- Faster data exchange rate compared to LF
- Support for cryptographic operations (depending on card*)
- Card specific command sets
  - MFU EV1: READ – 30 00 ← Reads page 0
  - MFDES: READ (fileId 01, offset 0, len 0) – 02 90 BD 00 00 07 01 00 00 00 00 00 00 00 ← Reads file 1

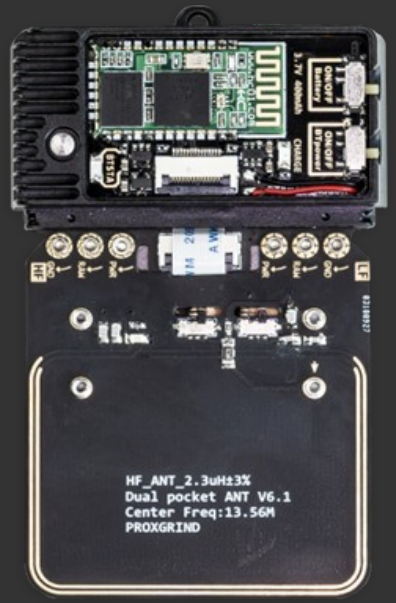
* Sometimes cloning can be as easy as cloning an LF tag. For example when only UID is used for authentication

Magic cards

- Cards that contain “magic commands”
  - Special commands that allow to e.g. read/write a certain memory region that would not be normally available
- Usually used by attackers
  - An attacker can e.g. change the UID of a card
- Some readers might do fingerprinting and try to reject magic cards
  - This is the reason why there exists one-time-write cards

Tools

- Flipper Zero
- Smartphone
- Chameleon Ultra
- ICopy-XS
- Proxmark



Flipper Zero

- Pros
  - Easy to use
  - Support for common HF and LF tags
- Cons
  - Not very good for low-level analysis
  - No support for e.g. more complicated attacks

Smartphone

- Pros
  - Most people already have a smartphone
  - Multiple easy to use tools available
- Cons
  - Hardware capabilities only for 13.56 MHz (at least on basic consumer phones)
  - Not very good for low-level analysis
  - No support for e.g. more complicated attacks

Chameleon Ultra

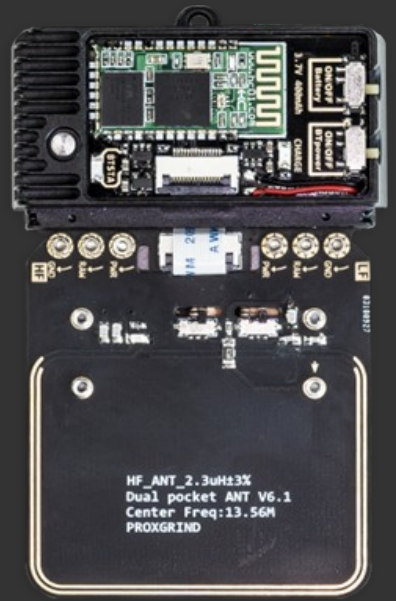
- Pros
  - Very small and easy to carry around
  - Good at emulating tags
  - Support for majority of HF and LF tags
- Cons
  - Not very good for low-level analysis

ICopy-XS

- Pros
  - Easy to use
  - Built on top of proxmark where advanced operations can be performed
  - Support for majority of HF and LF tags
- Cons
  - Device only supports ICopy-XS blank badges
  - Proprietary firmware and software

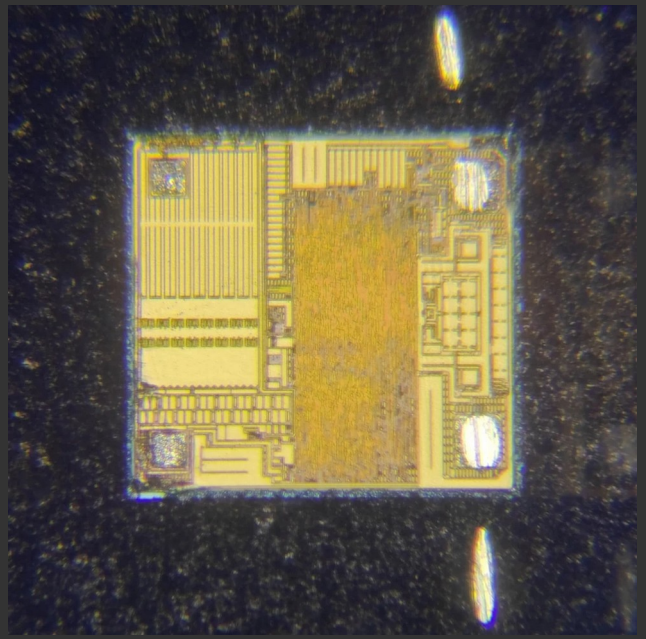
Proxmark

- Pros
  - Good for low-level analysis
  - Support for majority of HF and LF tags
  - Actively maintained open-source firmware + software (https://github.com/rfidresearchgroup/proxmark3)
- Cons
  - Requires technical knowledge to use



Background information

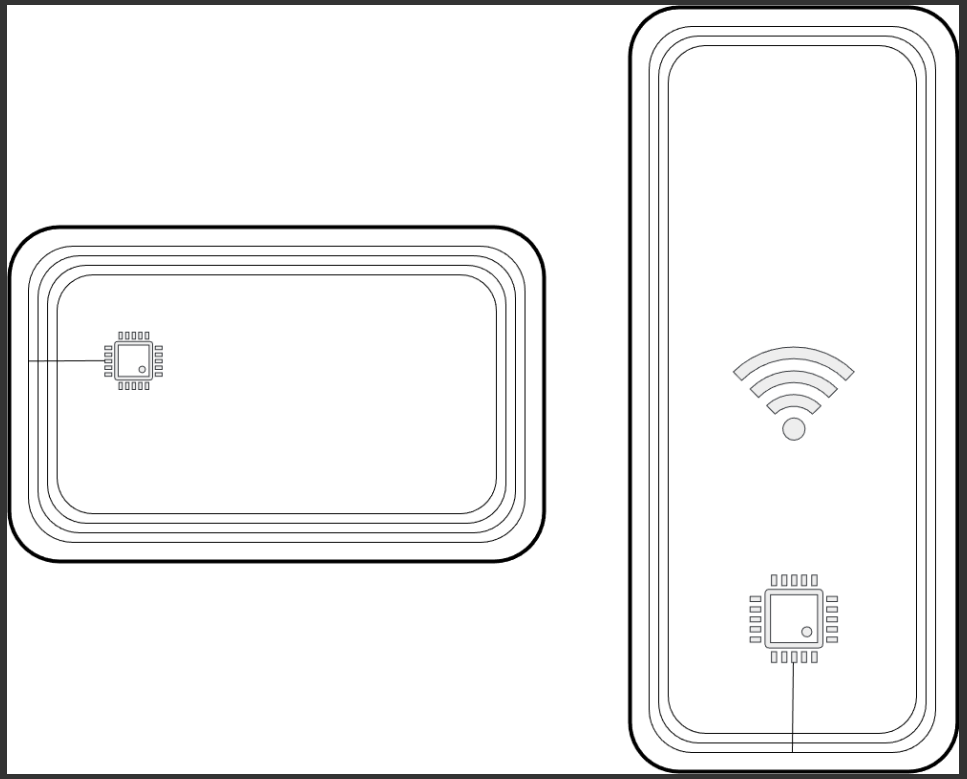
Physical Layer



- PICC (Card)
  - Passive technology = no battery
  - Power from PCD’s RF field
- PCD (Reader)
  - Active technology = has it’s own power source
  - Generates RF field for powering on PICC

Physical Layer

- PICC and PCD both contain
  - Antenna coils
  - Integrated circuits
  - Memory
- The PCD drives its antenna with an alternating current (AC)
  - The AC generates a time-varying magnetic field (Ampère-Maxwell’s law)



Physical Layer

- The changing magnetic field induces a voltage in the PICC’s antenna coil, resulting in current flow and powering the card (Faraday’s law of induction)
- The PICC transmits data by switching an additional load on and off its antenna, altering the coupling to the reader (load modulation)
- The load changes produce small variations in the PCD antenna current that are demodulated as data

PICC Memory Concepts

- Memory organization:
  - Pages: 4 bytes (e.g. MFU EV1)
  - Blocks: 4 pages (e.g. MFU C)
  - Sectors: 4 or 16 blocks (e.g. MFC1K & MFC4K)
  - Applications & files: Custom sized (e.g. MFDES)
- Card configuration & access conditions
- Manufacturer blocks
- OTP (One-Time Programmable) areas

ISO/IEC 14443

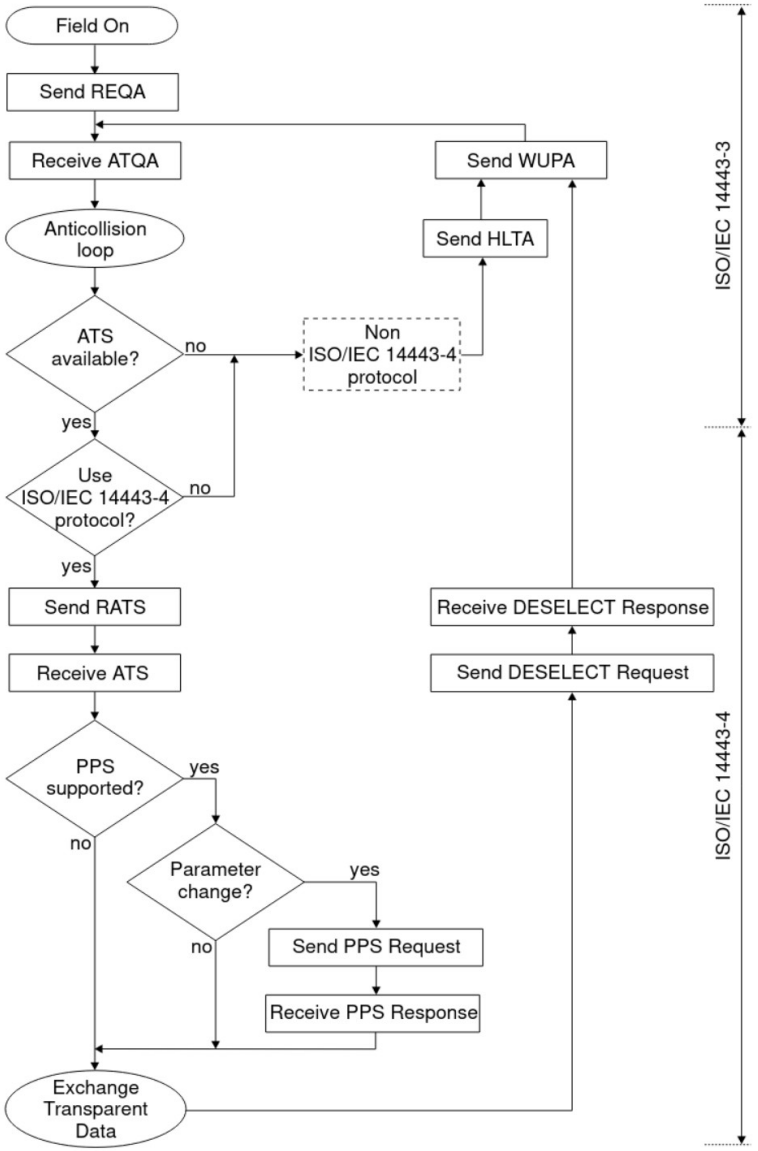
- Standard for contactless smart cards
- Part 1: Physical characteristics
- Part 2: RF power and signal interface
- Part 3: Initialization and anti-collision
- Part 4: Transmission protocol (Application Protocol Data Units)

ISO 14443A Init.

- Activation sequence defined in ISO/IEC 14443-3
  - Request
  - Anticollision loop
  - Select card
- Select Acknowledge (SAK) byte shall be checked for availability of an Answer-to-Select (ATS)
- Request for Answer-To-Select (RATS) if ATS available
- Protocol and Parameter Selection (PPS)
  (if supported and requested)

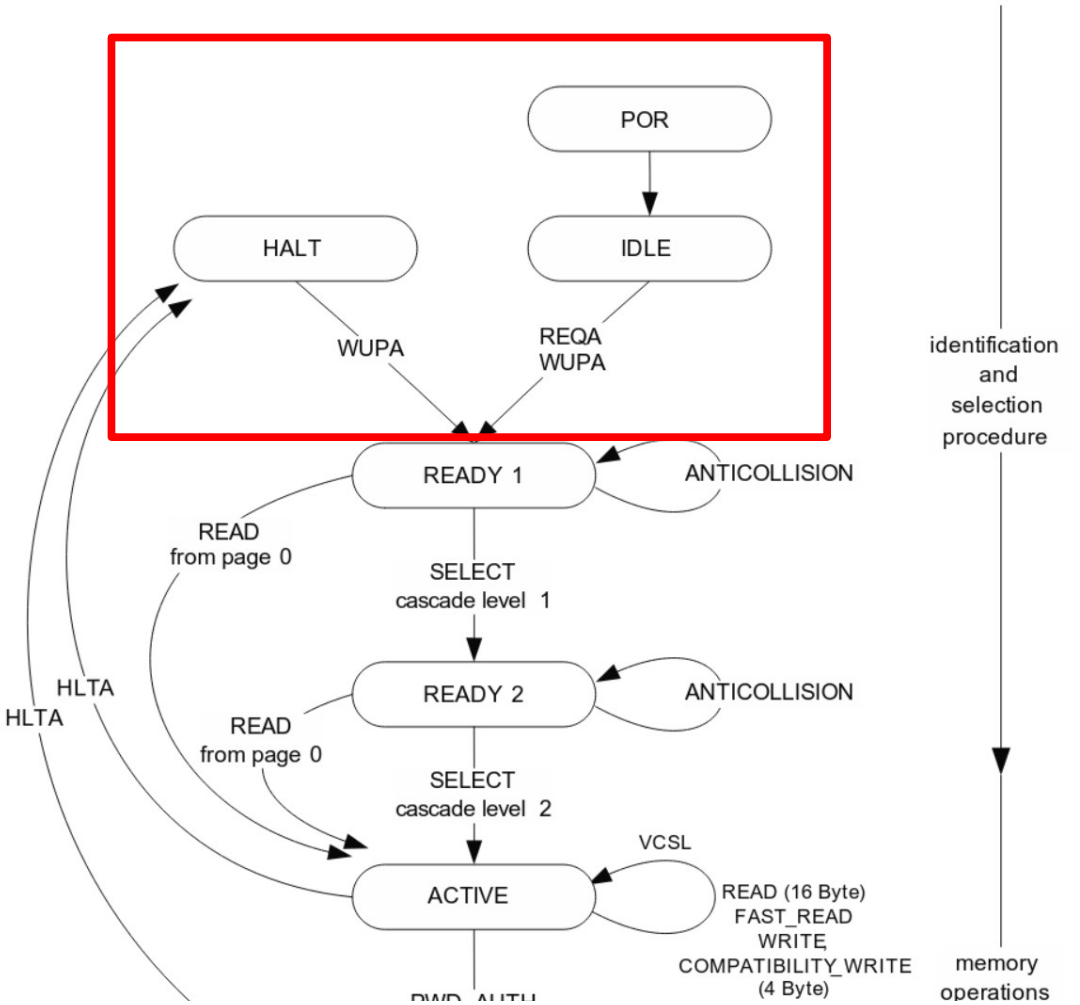
ISO 14443A Init.

- PCD sends REQA/WUPA
- PICC responds with ATQA
- Start anticollision loop
  - PCD performs card selection
  - PICC responds with SAK
- PCD sends RATS (If 14443-4 card)
- PICC responds with ATS
- Perform PPS (If supported & requested by PICC)
- Card activated



Example: MFU EV1

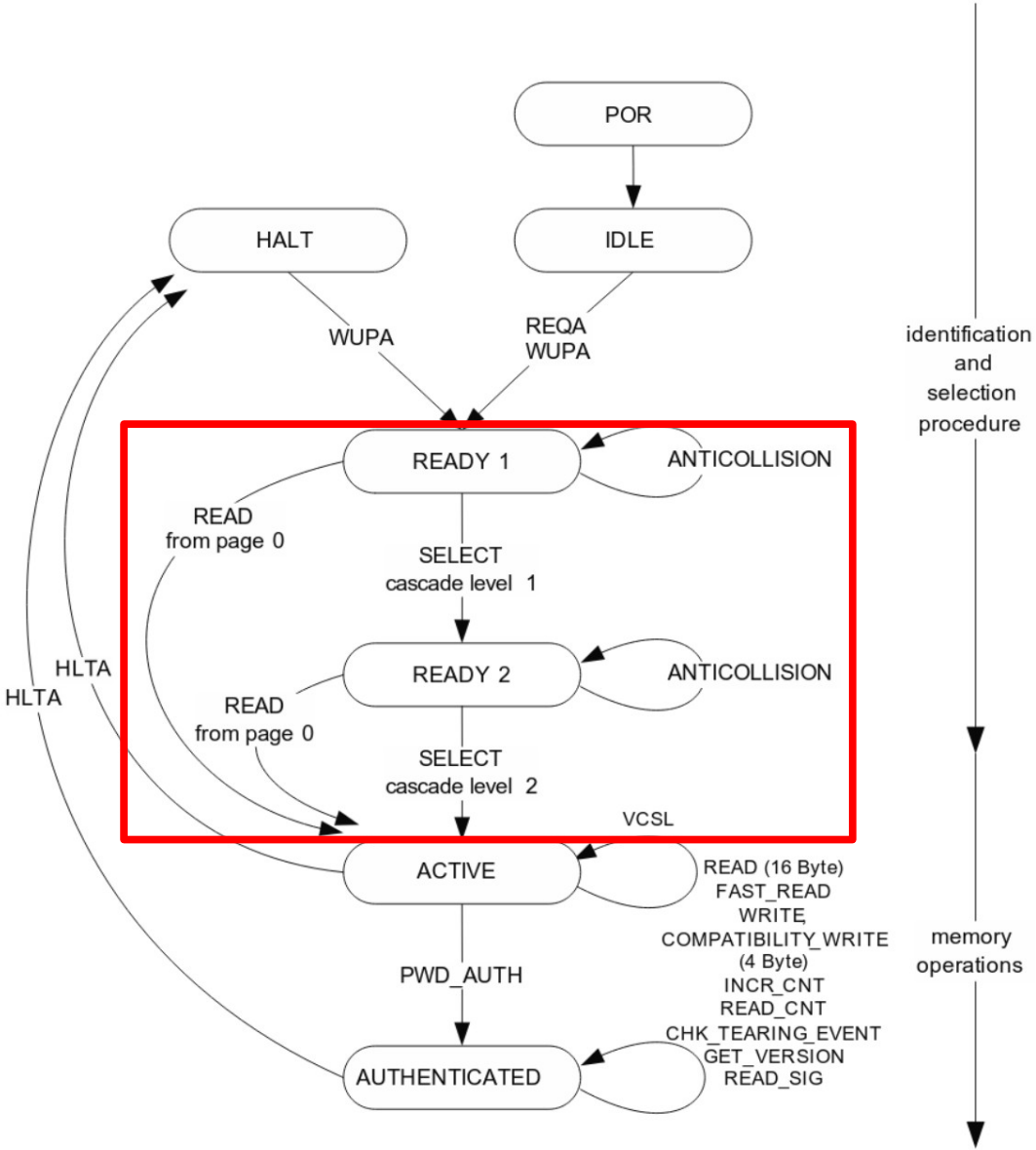
- Power-On Reset (POR)
  - PICC enters RF field and powers on
- IDLE
  - PICC waits for PCD to send REQA/WUPA
  - PICC responds with ATQA
  - The ATQA tells the reader PICC UID size and PICC type



Manufacturer	Product	ATQA	SAK	ATS (called ATR for contact smartcards)	UID length
NXP	MIFARE Ultralight	00 44	00		7 bytes

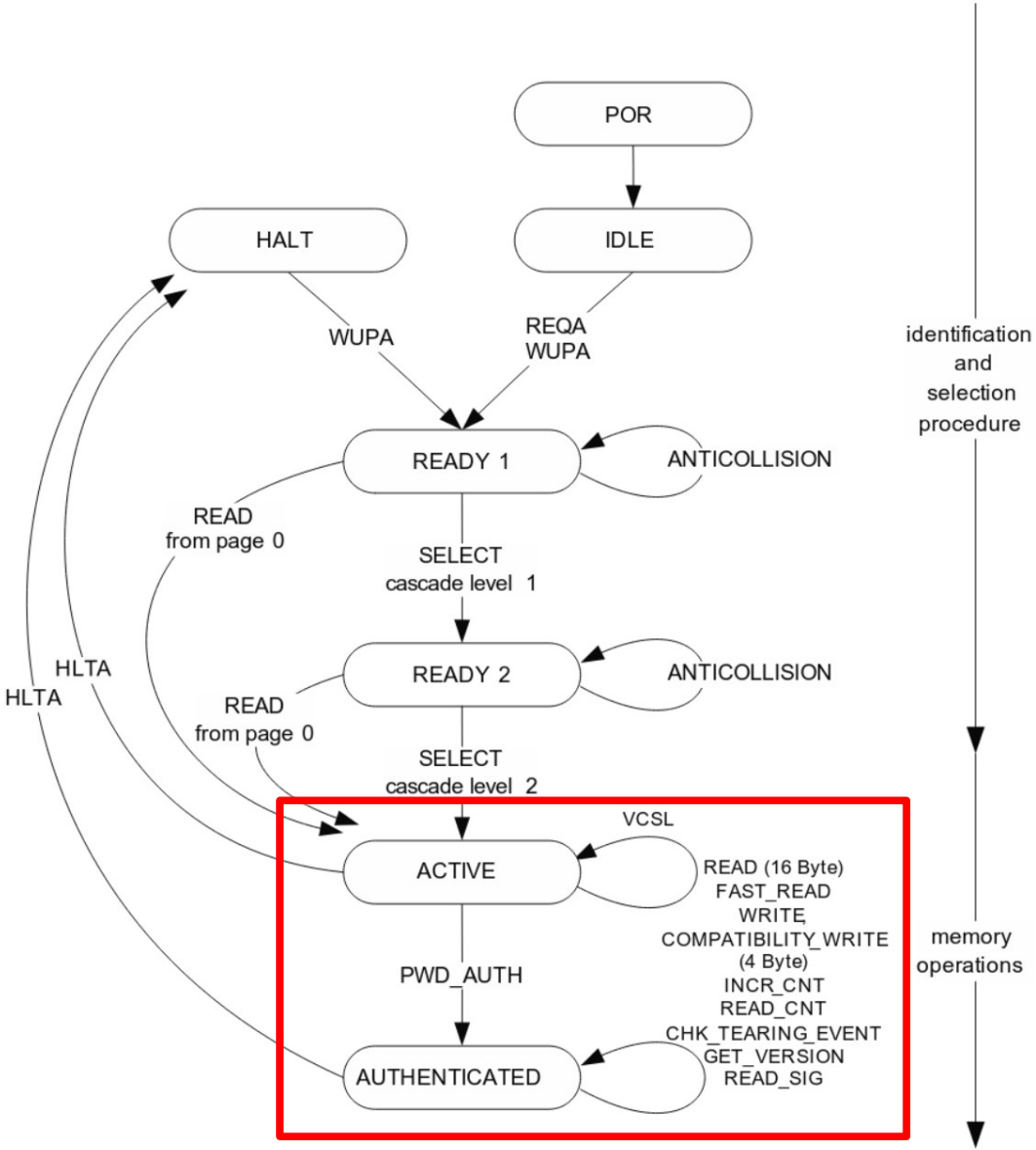
Example: MFU EV1

- READY 1
  - PCD performs anticollision (level 1) by requesting first part of PICC UID and a block check character (BCC)
  - PCD responds with UID to select
- READY 2
  - PCD performs anticollision (level 2) by requesting remaining UID bytes and another BCC
  - PCD responds with UID to select



Example: MFU EV1

- ACTIVE
  - PICC is now fully selected and in active state
  - PICC is ready to receive commands like READ & WRITE
  - Optionally PCD can send PWD_AUTH with a 4-byte password to move into AUTHENTICATED state
- AUTHENTICATED
  - Same as ACTIVE state but additional memory pages may become readable/writeable



Example: MFU EV1

- Communication as raw bytes seen through proxmark:

```
[usb] pm3 --> hf mfu info

[=] --- Tag Information ---------------------------
[+]       TYPE: MIFARE Ultralight EV1 48bytes (MF0UL1101) ( USCUID-UL )
[+]        UID: 04 11 22 2A BB 32 18
[+]     UID[0]: 04, NXP Semiconductors Germany
[+]       BCC0: BF ( ok )
[+]       BCC1: BB ( ok )
[+]   Internal: 48 ( default )
[+]       Lock: 00 00  - 0000000000000000
[+]        OTP: 00 00 00 00  - 00000000000000000000000000000000
```

```
[usb] pm3 --> hf mfu list
[+] Recorded activity ( 165 bytes )
[=] start = start of start frame. end = end of frame. src = source of transfer.
[=] ISO14443A - all times are in carrier periods (1/13.56MHz)

     Start |        End | Src | Data (! denotes parity error)                                    | CRC | Annotation
-----------+-----------+-----+-------------------------------------------------------------------+-----+-------------------
         0 |        992 | Rdr |52(7)                                                              |     | WUPA
      2116 |       4484 | Tag |44  00                                                             |     |
      7040 |       9504 | Rdr |93  20                                                             |     | ANTICOLL
     10564 |      16452 | Tag |88  04  11  22  BF                                                 |     |
     19584 |      30048 | Rdr |93  70  88  04  11  22  BF  B3  F9                                 |  ok | SELECT_UID
     31172 |      34692 | Tag |04  DA  17                                                         |  ok |
     35968 |      38432 | Rdr |95  20                                                             |     | ANTICOLL-2
     39492 |      45316 | Tag |2A  BB  32  18  BB                                                 |     |
     48384 |      58848 | Rdr |95  70  2A  BB  32  18  BB  FE  6F                                 |  ok | SELECT_UID-2
     59972 |      63556 | Tag |00  FE  51                                                         |  ok |
    363904 |     367520 | Rdr |60  F8  32                                                         |  ok | EV1 VERSION
    368580 |     380228 | Tag |00  04  03  01  01  00  0B  03  FD  F7                             |  ok |
```

Examples of HF cards

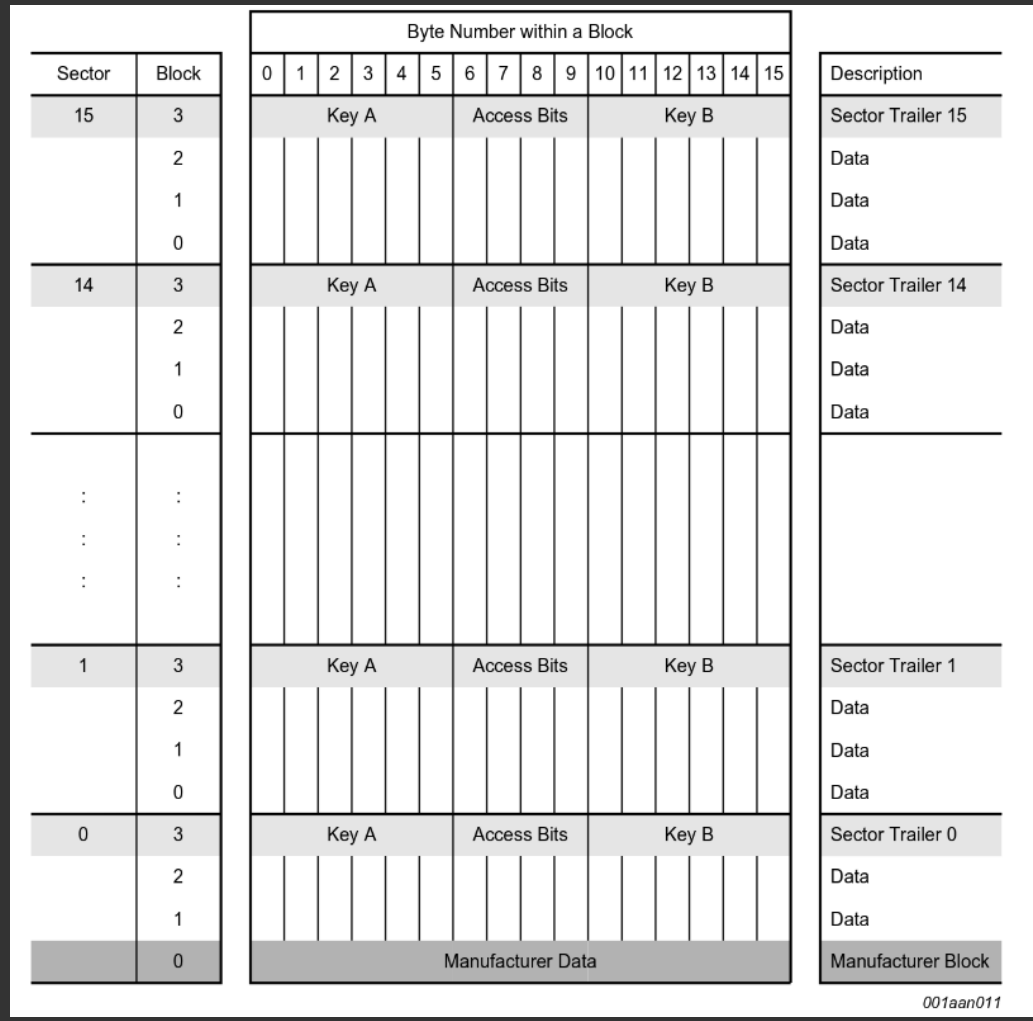
Mifare Classic 1K

- Memory organization
  - 16 sectors of 4 blocks (16 bytes per block)
  - Each sector contains (excluding sector 0)
    - 3 value blocks (user memory)
    - 1 trailer block (key A, access bits & key B for each sector)
- Uses encryption: Crypto1
  - Proprietary stream cipher with 48-bit key based on linear feedback shift registers (LFSR)
  - Broken since ~2008
  - *“Never roll your own crypto”*

```
[usb] pm3 --> hf mf dump --ns -k Projects/HelSecWorkshop/key_backups/mfc-key.bin
[+] Loaded binary key file `/home/sanduuz/Projects/HelSecWorkshop/key_backups/mfc-key.bin`
[=] Reading sector access bits...
[=] ................
[+] Finished reading sector access bits
[=] Dumping all blocks from card...
 🕓 Sector... 15 block... 3 ( ok )
[+] Succeeded in dumping all blocks

[+] time: 9 seconds


[=] -----+-----+-------------------------------------------------+----------------
[=]  sec | blk | data                                            | ascii
[=] -----+-----+-------------------------------------------------+----------------
[=]    0 |   0 | 6A 69 33 69 59 88 04 00 C8 09 00 20 00 00 00 22 | ji3iY...... ..."
[=]      |   1 | 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 | ................
[=]      |   2 | 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 | ................
[=]      |   3 | FF FF FF FF FF FF FF 07 80 69 FF FF FF FF FF FF | .........i......
[=]    1 |   4 | 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 | ................
[=]      |   5 | 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 | ................
[=]      |   6 | 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 | ................
[=]      |   7 | FF FF FF FF FF FF FF 07 80 69 FF FF FF FF FF FF | .........i......
[=]    2 |   8 | 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 | ................
[=]      |   9 | 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 | ................
[=]      |  10 | 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 | ................
[=]      |  11 | FF FF FF FF FF FF FF 07 80 69 FF FF FF FF FF FF | .........i......
[=]    3 |  12 | 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 | ................
[=]      |  13 | 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 | ................
[=]      |  14 | 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 | ................
[=]      |  15 | FF FF FF FF FF FF FF 07 80 69 FF FF FF FF FF FF | .........i......
[=]    4 |  16 | 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 | ................
[=]      |  17 | 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 | ................
[=]      |  18 | 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 | ................
[=]      |  19 | FF FF FF FF FF FF FF 07 80 69 FF FF FF FF FF FF | .........i......
[=]    5 |  20 | 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 | ................
[=]      |  21 | 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 | ................
[=]      |  22 | 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 | ................
[=]      |  23 | FF FF FF FF FF FF FF 07 80 69 FF FF FF FF FF FF | .........i......
[=]    6 |  24 | 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 | ................
[=]      |  25 | 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 | ................
[=]      |  26 | 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 | ................
[=]      |  27 | FF FF FF FF FF FF FF 07 80 69 FF FF FF FF FF FF | .........i......
```



Mifare Classic 1K

- Attacks/problems
  - Misconfigurations: E.g. using default keys
  - Darkside: Parity-bit oracle + weak nonce generation
    - Part of keystream can be inferred based on failed authentications
  - Bruteforce: Plain old bruteforce
    - Key only 48-bits
    - Can be performed against pre-recorded authentication exchanges

Mifare Classic 1K

- Attacks/problems
  - Nested: Same keystream reused across authentications
    - Authenticate to a block with known key
    - Reauthenticate to same block within an encrypted session (hence, name nested)
    - Compute number of LFSR shifts to guess the next nonce to calculate keys and try to authenticate to a different block
  - HardNested: Optimized version of nested attack
    - Doesn’t require initial key for attack
  - StaticNested: Older versions used repeating nonces
    - Keystream recovery trivial when combined with nested attack

Mifare Ultralight EV1

- Memory organization:
  - 20 pages (4 bytes per page)
    - Pages 0-2: Manufacturer data and lock bytes
    - Page 3: OTP area
    - Pages 4-15: User memory (48 bytes)
    - Pages 16-19: Card configuration
- 3DES based password auth (4 bytes)

```
[=] ------------------------------------
[=] block#   | data        |lck| ascii
[=] ---------+-------------+---+------
[=]   0/0x00 | 04 11 22 BF |   | ..".
[=]   1/0x01 | 2A BB 32 18 |   | *.2.
[=]   2/0x02 | BB 48 00 00 |   | .H..
[=]   3/0x03 | 00 00 00 00 | 0 | ....
[=]   4/0x04 |             | 0 |
[=]   5/0x05 |             | 0 |
[=]   6/0x06 |             | 0 |
[=]   7/0x07 |             | 0 |
[=]   8/0x08 |             | 0 |
[=]   9/0x09 |             | 0 |
[=]  10/0x0A | 00 00 00 00 | 0 | ....
[=]  11/0x0B | 00 00 00 00 | 0 | ....
[=]  12/0x0C | 00 00 00 00 | 0 | ....
[=]  13/0x0D | 00 00 00 00 | 0 | ....
[=]  14/0x0E |             | 0 |
[=]  15/0x0F | 00 00 00 00 | 0 | ....
[=]  16/0x10 | 00 00 00 04 | ? | ....
[=]  17/0x11 | 80 05 00 00 | ? | ....
[=]  18/0x12 |             | ? |
[=]  19/0x13 | 00 00 00 00 | ? | ....
[=] --------------------------------
```

Page Adr Dec	Page Adr Hex	Byte number within a page 0	1	2	3	Description
0	0h	serial number				Manufacturer data and lock bytes
1	1h	serial number				
2	2h	serial number	internal	lock bytes		
3	3h	OTP	OTP	OTP	OTP	One Time Programmable
4	4h	user memory				User memory pages
5	5h					
...	...					
14	Eh					
15	Fh					
16	10h	CFG0				Configuration pages
17	11h	CFG1				
18	12h	PWD				
19	13h	PACK		RFUI		
		One-Way counters [1]				Counter pages

aaa-006275

1. counter pages are only accessible with READ_CNT and INCR_CNT commands

Mifare Ultralight EV1

- Attacks/problems
  - Misconfigurations: E.g. incorrect card configuration (data can be read without authentication)
  - Sniffing: The 3DES password is sent unencrypted from PCD to PICC
  - Data on card not encrypted: It might be possible to read the data on the card if password can be somehow bypassed

Mifare Ultralight C

- Memory organization:
  - Similar to Ultralight EV1 with more user memory
- 3DES based password auth (16 bytes)
  - This time password is not sent in plaintext, instead three-pass protocol is used

Reader		Card
	AUTH (02 0A 00) $\longrightarrow$	$n_C \in_R \{0,1\}^{64}$
$n_R \in_R \{0,1\}^{64}$ $b_1 = Enc_{K_R}(n_R)$ $r_0 = Dec_{K_R}(b_0)$ $r_1 = RotLeft_8(r_0)$	$\longleftarrow$ b_0	$b_0 = Enc_{K_C}(n_C)$
$b_2 = Enc_{K_R}(r_1 \oplus b_1)$	$\xrightarrow{b_1, b_2}$	$r_2 = Dec_{K_C}(b_2)$ $n'_C = RotRight_8(r_2 \oplus b_1)$
	$\longleftarrow$ ERROR (02 AE)	if $n'_C \neq n_C$ else if $n'_C = n_C$ $r_3 = Dec_{K_C}(b_1)$ $r_4 = RotRight_8(r_3)$
$r_5 = Dec_{K_R}(b_3)$ $n'_R = RotLeft_8(r_5)$ verify $n'_R \stackrel{?}{=} n_R$	$\longleftarrow$ b_3	$b_3 = Enc_{K_C}(r_4)$

```
[=] ------------------------------------
[=] block#   | data        |lck| ascii
[=] ---------+-------------+---+------
[=]   0/0x00 | 04 15 91 08 |   | ....
[=]   1/0x01 | 00 02 F2 56 |   | ...V
[=]   2/0x02 | A6 48 00 00 |   | .H..
[=]   3/0x03 | 00 00 00 00 | 0 | ....
[=]   4/0x04 | 00 00 00 00 | 0 | ....
[=]   5/0x05 | 00 00 00 00 | 0 | ....
[=]   6/0x06 | 00 00 00 00 | 0 | ....
[=]   7/0x07 | 00 00 00 00 | 0 | ....
[=]   8/0x08 | 00 00 00 00 | 0 | ....
[=]   9/0x09 | 00 00 00 00 | 0 | ....
[=]  10/0x0A | 00 00 00 00 | 0 | ....
[=]  11/0x0B | 00 00 00 00 | 0 | ....
[=]  12/0x0C | 00 00 00 00 | 0 | ....
[=]  13/0x0D | 00 00 00 00 | 0 | ....
[=]  14/0x0E |             | 0 |
[=]  15/0x0F |             | 0 |
[=]  16/0x10 |             | ? |
[=]  17/0x11 |             | ? |
[=]  18/0x12 |             | ? |
[=]  19/0x13 |             | ? |
[=]  20/0x14 |             | ? |
[=]  21/0x15 | 00 00 00 00 | ? | ....
[=]  22/0x16 | 00 00 00 00 | ? | ....
[=]  23/0x17 | 00 00 00 00 | ? | ....
[=]  24/0x18 | 00 00 00 00 | ? | ....
[=]  25/0x19 | 00 00 00 00 | ? | ....
[=]  26/0x1A | 00 00 00 00 | ? | ....
```

Table 5. Memory organization

Page address		Byte number			
Decimal	Hex	0	1	2	3
0	00h	serial number			
1	01h	serial number			
2	02h	serial number	internal	lock bytes	lock bytes
3	03h	OTP	OTP	OTP	OTP
4 to 39	04h to 27h	user memory	user memory	user memory	user memory
40	28h	lock bytes	lock bytes	-	-
41	29h	16-bit counter	16-bit counter	-	-
42	2Ah	authentication configuration			
43	2Bh	authentication configuration			
44 to 47	2Ch to 2Fh	authentication key			

Mifare Ultralight C

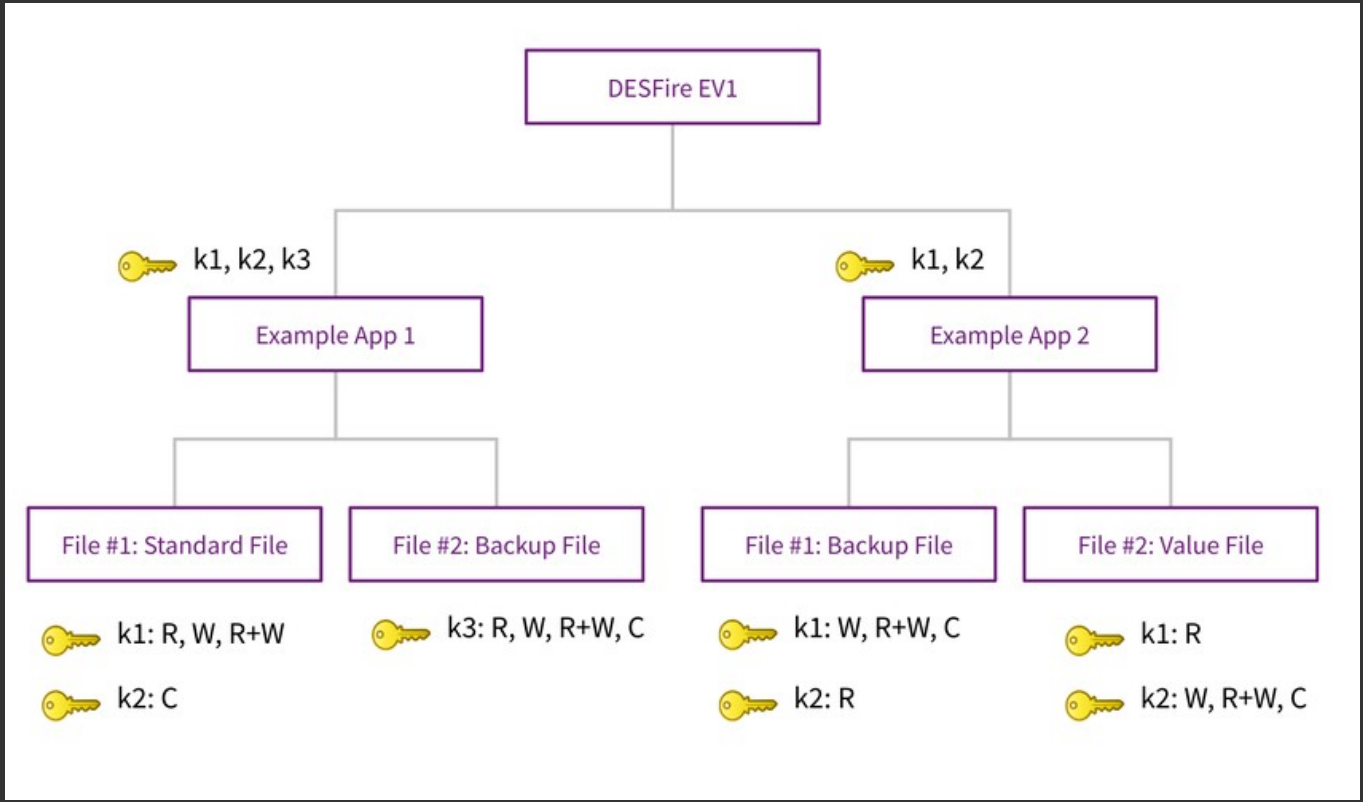
- Attacks/problems
  - Misconfigurations: E.g. incorrect card configuration (data can be read without authentication)
  - Sniffing: Even though authentication is performed using three-pass protocol, the data transmitted after authentication is still sent in plaintext
  - First practical attacks published just 4 days ago (2026-01-25)! (https://breakmeifyoucan.com/)
    - Keyspace reduction and relay attacks

Mifare DESFire

- Memory organization:
  - Applications
    - Files
      - Standard data file, backup data file, value file, linear record file, cyclic linear record file
- Supports strong AES encryption
  - Application & file specific keys
  - Files have different communication modes
    - Plain, MAC, Enciphered

```
[usb] pm3 --> hf mfdes getaids

[+] --- AID list ( 8 found )
[+] F9AE22
[+] 548190
[+] EB8CA5
[+] 48BD05
[+] 8B1E2D
[+] EB10BA
[+] 0C8F31
[+] 031337
```




```
[usb] pm3 --> hf mfdes lsfiles --aid F9AE22
[!!] 🚨 Desfire GetFileISOIDList command error. Result: -20
[!] ⚠️ ISO ID list returned no data
[=] ---------------------------------------- File list ----------------------------------------------------
[+]  ID |ISO ID| File type            | Mode  | Rights: raw, r w rw ch    | File settings
[+] ---------------------------------------------------------------------------------------------------
[+]  00 |      | 0x02 Value           | Plain | eeee, free free free free | Value [0 .. 2147483647] lim cred: 0x00 (0 [0x0])
[+]  01 |      | 0x00 Standard data   | MAC   | e000, free key0 key0 key0 | Size 6 / 0x6
[usb] pm3 -->
```

Mifare DESFire

- Attacks/problems
  - Misconfigurations: E.g. authentication file communication mode set only as plain/MAC (Can be read without authentication)
  - No known practical attacks*

* This does not mean the card does not have any vulnerabilities. It’s just a matter of when, not if.

Hands-on challenges

Proxmark Quick Start Guide

```
[usb] pm3 --> help

help            Use `<command> help` for details of a command
prefs           { Edit client/device preferences... }
-------         ---------------------- Technology ----------------------
analyse         { Analyse utils... }
data            { Plot window / data buffer manipulation... }
emv             { EMV ISO-14443 / ISO-7816... }
hf              { High frequency commands... }
hw              { Hardware commands... }
lf              { Low frequency commands... }
mem             { Flash memory manipulation... }
mqtt            { MQTT commmands... }
nfc             { NFC commands... }
piv             { PIV commands... }
reveng          { CRC calculations from RevEng software... }
smart           { Smart card ISO-7816 commands... }
script          { Scripting commands... }
trace           { Trace manipulation... }
wiegand         { Wiegand format manipulation... }
-------         ---------------------- General ----------------------
auto            Automated detection process for unknown tags
clear           Clear screen
hints           Turn hints on / off
msleep          Add a pause in milliseconds
rem             Add a text line in log file
quit
exit            Exit program

[usb] pm3 -->
```

HF and LF commands

```
[usb] pm3 --> hf help

--------          ---------------------- High Frequency ----------------------
14a               { ISO14443A RFIDs...                  }
14b               { ISO14443B RFIDs...                  }
15                { ISO15693 RFIDs...                   }
cipurse           { Cipurse transport Cards...          }
epa               { German Identification Card...       }
emrtd             { Machine Readable Travel Document... }
felica            { ISO18092 / FeliCa RFIDs...          }
fido              { FIDO and FIDO2 authenticators...    }
fudan             { Fudan RFIDs...                      }
gallagher         { Gallagher DESFire RFIDs...          }
iclass            { ICLASS RFIDs...                     }
ict               { ICT MFC/DESfire RFIDs...            }
jooki             { Jooki RFIDs...                      }
ksx6924           { KS X 6924 (T-Money, Snapper+) RFIDs }
legic             { LEGIC RFIDs...                      }
lto               { LTO Cartridge Memory RFIDs...       }
mf                { MIFARE RFIDs...                     }
mfp               { MIFARE Plus RFIDs...                }
mfu               { MIFARE Ultralight RFIDs...          }
mfdes             { MIFARE Desfire RFIDs...             }
ntag424           { NXP NTAG 4242 DNA RFIDs...          }
saflok            { Saflok MFC RFIDs...                 }
seos              { SEOS RFIDs...                       }
st25ta            { ST25TA RFIDs...                     }
tesla             { TESLA Cards...                      }
texkom            { Texkom RFIDs...                     }
thinfilm          { Thinfilm RFIDs...                   }
topaz             { TOPAZ (NFC Type 1) RFIDs...         }
vas               { Apple Value Added Service...        }
waveshare         { Waveshare NFC ePaper...             }
xerox             { Fuji/Xerox cartridge RFIDs...       }
-----------       -------------------- General --------------------
help              This help
list              List protocol data in trace buffer
plot              Plot signal
tune              Continuously measure HF antenna tuning
search            Search for known HF tags
sniff             Generic HF Sniff

[usb] pm3 -->
```

```
[usb] pm3 --> lf help

help              This help
-----------       -------------- Low Frequency --------------
awid              { AWID RFIDs...              }
cotag             { COTAG CHIPs...             }
destron           { FDX-A Destron RFIDs...     }
em                { EM CHIPs & RFIDs...        }
fdxb              { FDX-B RFIDs...             }
gallagher         { GALLAGHER RFIDs...         }
gproxii           { Guardall Prox II RFIDs...  }
hid               { HID Prox RFIDs...          }
hitag             { Hitag CHIPs...             }
idteck            { Idteck RFIDs...            }
indala            { Indala RFIDs...            }
io                { ioProx RFIDs...            }
jablotron         { Jablotron RFIDs...         }
keri              { KERI RFIDs...              }
motorola          { Motorola Flexpass RFIDs... }
nedap             { Nedap RFIDs...             }
nexwatch          { NexWatch RFIDs...          }
noralsy           { Noralsy RFIDs...           }
pac               { PAC/Stanley RFIDs...       }
paradox           { Paradox RFIDs...           }
pcf7931           { PCF7931 CHIPs...           }
presco            { Presco RFIDs...            }
pyramid           { Farpointe/Pyramid RFIDs... }
securakey         { Securakey RFIDs...         }
ti                { TI CHIPs...                }
t55xx             { T55xx CHIPs...             }
viking            { Viking RFIDs...            }
visa2000          { Visa2000 RFIDs...          }
-----------       --------------------- General ---------------------
config            Get/Set config for LF sampling, bit/sample, decimation, frequency
cmdread           Modulate LF reader field to send command before read
read              Read LF tag
search            Read and Search for valid known tag
sim               Simulate LF tag from buffer
simask            Simulate ASK tag
simfsk            Simulate FSK tag
simpsk            Simulate PSK tag
simbidir          Simulate LF tag (with bidirectional data transmission between reader and tag)
sniff             Sniff LF traffic between reader and tag
tune              Continuously measure LF antenna tuning

[usb] pm3 -->
```

Sniffing

```
[usb] pm3 --> hf 14a sniff help
hf 14a sniff: unexpected argument "help"
[!] ⚠️ Try 'hf 14a sniff --help' for more information.

[usb] pm3 --> hf 14a sniff --help

Sniff the communication between reader and tag
Use `hf 14a list` to view collected data.

usage:
    hf 14a sniff [-hcri]

options:
    -h, --help                    This help
    -c, --card                    triggered by first data from card
    -r, --reader                  triggered by first 7-bit request from reader (REQ, WUP)
    -i, --interactive             Console will not be returned until sniff finishes or is aborted

examples/notes:
     hf 14a sniff -c -r

[usb] pm3 -->
```

Sniffing

```
[usb] pm3 --> hf mfu list --help

Alias of `trace list -t 14a -c` with selected protocol data to annotate trace buffer
You can load a trace from file (see `trace load -h`) or it be downloaded from device by default
It accepts all other arguments of `trace list`. Note that some might not be relevant for this specific protocol

usage:
    hf 14a list [-h1crux] [--frame] [-f <fn>]

options:
    -h, --help                      This help
    -1, --buffer                    use data from trace buffer
    --frame                         show frame delay times
    -c                              mark CRC bytes
    -r                              show relative times (gap and duration)
    -u                              display times in microseconds instead of clock cycles
    -x                              show hexdump to convert to pcap(ng)
                                    or to import into Wireshark using encapsulation type "ISO 14443"
    -f, --file <fn>                 filename of dictionary

examples/notes:
    hf 14a list --frame             -> show frame delay times
    hf 14a list -1                  -> use trace buffer

[usb] pm3 -->
```

Emulating

```
[usb] pm3 --> hf 14a sim -h

Simulate ISO/IEC 14443 type A tag with 4,7 or 10 byte UID
Use type 7 for Mifare Ultralight EV1, Amiibo (NTAG215 pack 0x8080)

usage:
    hf 14a sim [-hxv] -t <1-14>  [-u <hex>] [-n <dec>] [--sk] [--1a1 <hex>] [--1a2 <hex>]

options:
    -h, --help                      This help
    -t, --type <1-14>               Simulation type to use
    -u, --uid <hex>                 <4|7|10> hex bytes UID
    -n, --num <dec>                 Exit simulation after <numreads> blocks have been read by reader. 0 = infinite
    -x                              Performs the 'reader attack', nr/ar attack against a reader
    --sk                            Fill simulator keys from found keys
    -v, --verbose                   verbose output
    --1a1 <hex>                     <8|16> hex bytes ULC/ULAES Auth reply step1: ek(RndB)
    --1a2 <hex>                     <8|16> hex bytes ULC/ULAES Auth reply step2: ek(RndA')

examples/notes:
    hf 14a sim -t 1 --uid 11223344       -> MIFARE Classic 1k
    hf 14a sim -t 2                      -> MIFARE Ultralight
    hf 14a sim -t 3                      -> MIFARE Desfire
    hf 14a sim -t 4                      -> ISO/IEC 14443-4
    hf 14a sim -t 5                      -> MIFARE Tnp3xxx
    hf 14a sim -t 6                      -> MIFARE Mini
    hf 14a sim -t 7                      -> MFU EV1 / NTAG 215 Amiibo
    hf 14a sim -t 8                      -> MIFARE Classic 4k
    hf 14a sim -t 9                      -> FM11RF005SH Shanghai Metro
    hf 14a sim -t 10                     -> ST25TA IKEA Rothult
    hf 14a sim -t 11                     -> Javacard (JCOP)
    hf 14a sim -t 12                     -> 4K Seos card
    hf 14a sim -t 13                     -> MIFARE Ultralight C
    hf 14a sim -t 14                     -> MIFARE Ultralight AES
```

Emulating

```
[usb] pm3 --> hf mfu sim -h

Simulate MIFARE Ultralight family type based upon
ISO/IEC 14443 type A tag with 4,7 or 10 byte UID
from emulator memory.  See `hf mfu eload` first.
The UID from emulator memory will be used if not specified.
See `hf 14a sim -h` to see available types. You want 2, 7, 13 or 14 usually.

usage:
    hf mfu sim [-hv] -t <1..14>  [-u <hex>] [-n <dec>] [--1a1 <hex>] [--1a2 <hex>]

options:
    -h, --help                      This help
    -t, --type <1..14>              Simulation type to use
    -u, --uid <hex>                 <4|7|10> hex bytes UID
    -n, --num <dec>                 Exit simulation after <numreads> blocks. 0 = infinite
    -v, --verbose                   Verbose output
    --1a1 <hex>                     <8|16> hex bytes ULC/ULAES Auth reply step1: ek(RndB)
    --1a2 <hex>                     <8|16> hex bytes ULC/ULAES Auth reply step2: ek(RndA')

examples/notes:
    hf mfu sim -t 2 --uid 11223344556677             -> MIFARE Ultralight
    hf mfu sim -t 7 --uid 11223344556677 -n 5        -> MFU EV1 / NTAG 215 Amiibo
    hf mfu sim -t 7                                  -> MFU EV1 / NTAG 215 Amiibo
    hf mfu sim -t 13                                 -> MIFARE Ultralight C
    hf mfu sim -t 14                                 -> MIFARE Ultralight AES
```

Emulating

```
[usb] pm3 --> hf mfu eload -h

Load emulator memory with data from (bin/eml/json) dump file


usage:
    hf mfu eload [-hv] -f <fn> [-q <dec>]

options:
    -h, --help                      This help
    -f, --file <fn>                 Specify a filename for dump file
    -q, --qty <dec>                 Number of blocks to load from eml file
    -v, --verbose                   verbose output

examples/notes:
    hf mfu eload -f hf-mfu-04010203040506.bin
    hf mfu eload -f hf-mfu-04010203040506.bin -q 57        -> load 57 blocks from myfile

[usb] pm3 -->
```

Challenges

- Practice hands-on skills in small groups
- 5 Challenges
  - Different aspects of attacks
- Objectives
  - Learn to use different tools
  - Find flags in PICC memory (Flag prefix: HELSEC_)
  - Clone/simulate PICCs
  - Analyze communication protocols
  - Have fun!
- Collaborate – Ask for help and help eachother!

Challenges

- **Do NOT use magic features** – Legitimate cards in the wild are not magic cards (usually)
  - Commands prefixed with c or g, e.g. csetuid/gsetuid
  - Commands with gen2/gen3/gdm in them

- **Do NOT write to the cards** – This will most likely break functionality of the challenge (and in the worst case might even brick a card if e.g. invalid configuration is written)
  - Includes e.g. write/wrbl/wipe commands

Challenges

- Practical tips:
  - Try to gain an understanding of the system
    - What information does the PCD try to access?
  - Gather information of the card (like card type, card configuration, etc.)
- If you don’t get a proper read (lots of errors, no information, etc.):
  - Try to alter the distance of the card from the reader up to few centimeters. This is most likely caused due to poor coupling with PCD and PICC.

Thank you!

(Next: small break and then hacking begins)

Cheatsheet

- Do **NOT** use magic commands (e.g. csetblk, gen3uid, gsetblk, gdmsetblk)
- Do **NOT** write to cards (e.g. wrbl, wipe, restore)
- If you accidentally exit from pm3 shell run: ~/proxmark3/pm3
- Useful proxmark commands (remember you can always use --help):
  - Search for tags: lf search / hf search
  - HF card specific commands: hf mf / hf mfu / hf mfdes
  - Sniffing: lf sniff / hf 14a sniff
  - Listing sniffs: hf mf list / hf mfu list / hf mfdes list
  - Preparing emulator: hf mf eload / hf mfu eload
  - Simulating card: hf mf sim / hf mfu sim